

CSE 465

Mobile Computing

2. On-Device AI

Taesik Gong, taesik.gong@unist.ac.kr

[Ubiquitous Artificial Intelligence Lab](#),

Department of Computer Science and Engineering & Artificial Intelligence Graduate School,
Ulsan National Institute of Science and Technology

Today's Lecture

- On-Device AI Overview
 - On-Device System Constraints
 - On-Device AI Hardware
 - On-Device AI Software Ecosystem
- We focus on the big picture and key concepts rather than low-level details

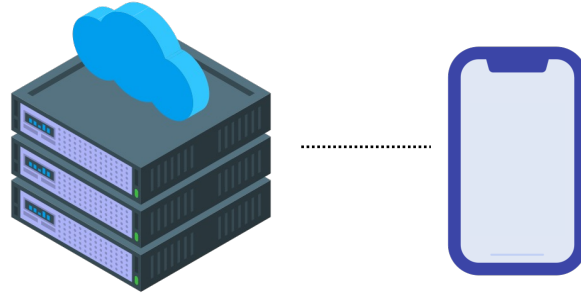
The AI Computing Paradigm Shift

- **Cloud-Centric AI:** Relies on **massive data centers** (e.g., A100/H100 clusters). Offers virtually infinite compute and memory, but suffers from high latency, massive bandwidth costs, and single points of failure.
- **Edge Computing:** Pushes computation to **intermediate nodes** (base stations, edge routers). Reduces backbone internet traffic and lowers latency, but still requires a reliable wireless hop (4G/5G/WiFi).
- **On-Device AI :** Shifts the entire inference pipeline directly to the **end-user's endpoint** (smartphones, wearables). Computation occurs exactly where the physical sensor data is generated.

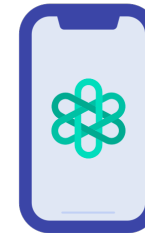
On-Device AI

Running AI models on edge devices without cloud servers

Cloud-based AI

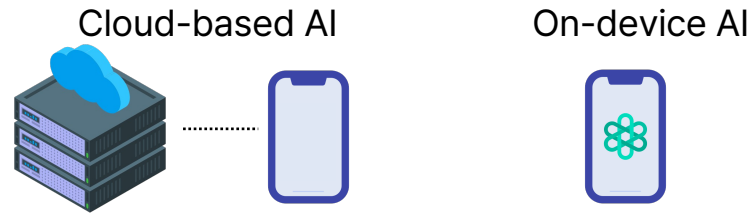


On-device AI

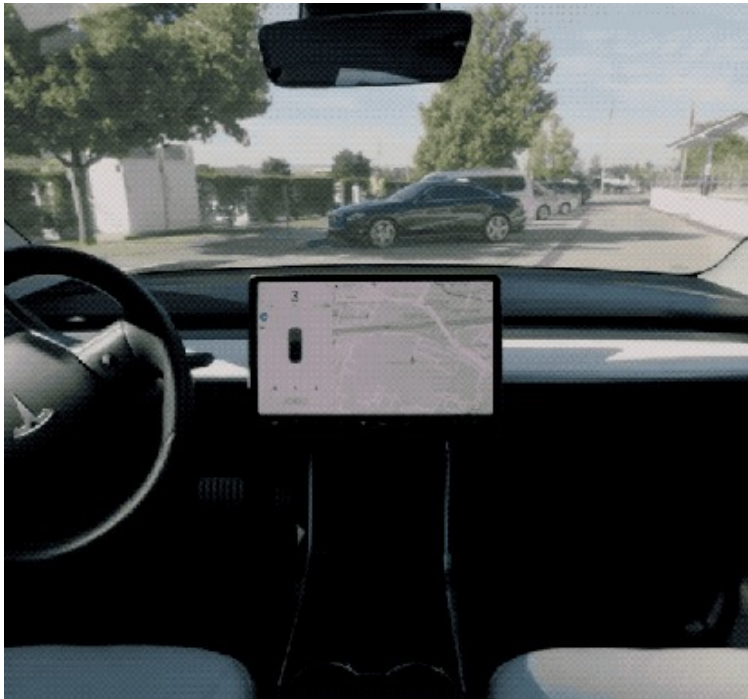


On-Device AI

Running AI models on edge devices without cloud servers



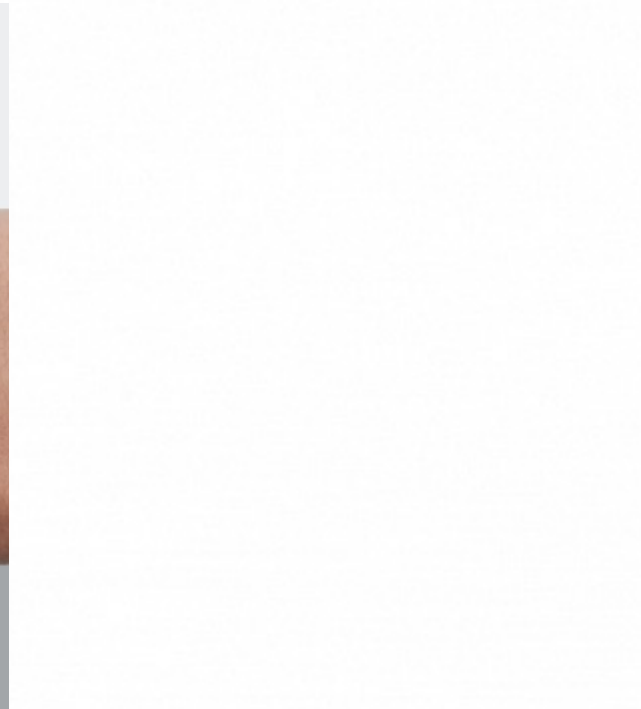
Tesla Autopilot



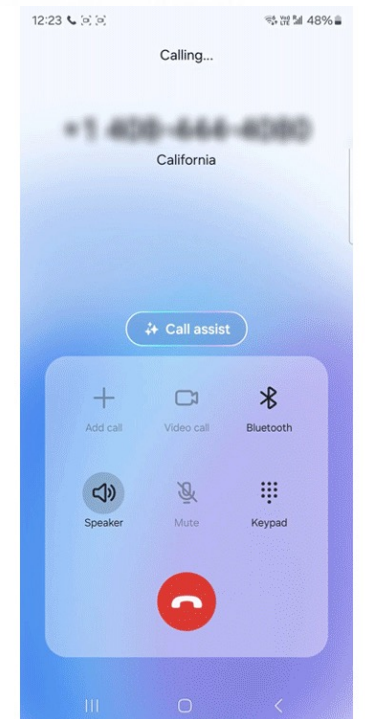
Apple Watch Siri



Google Gemini

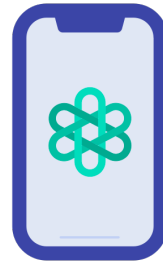


Samsung Live Translate

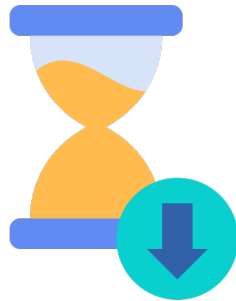


Why Is On-Device AI Important?

Benefits of on-device AI



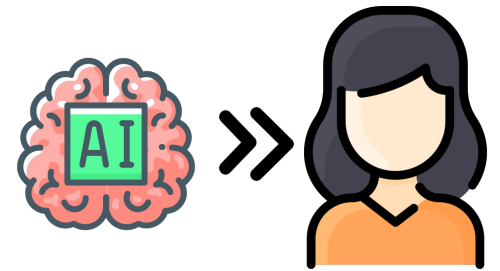
Privacy & Security



Reduced Latency



Zero Networking Cost

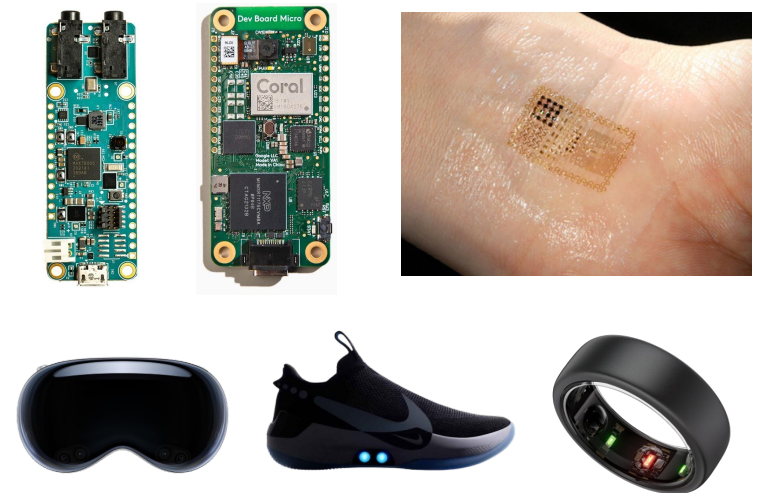
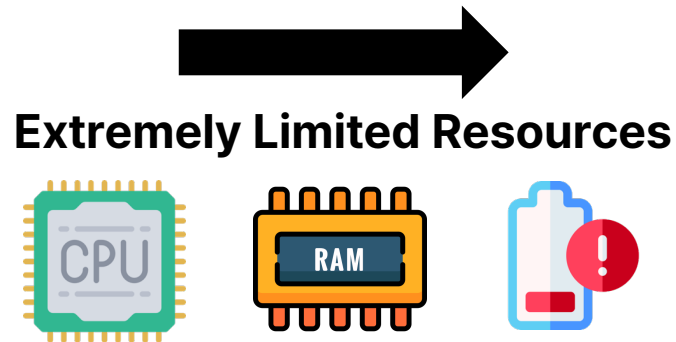
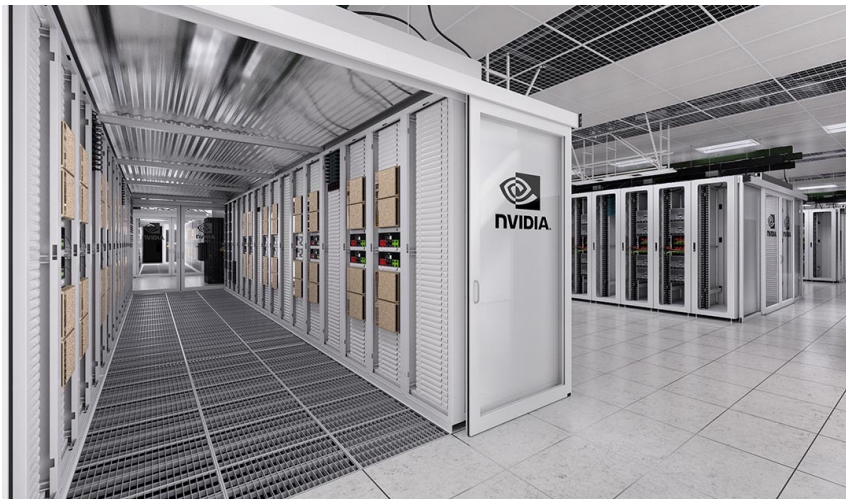


Personalization

On-Device System Constraints

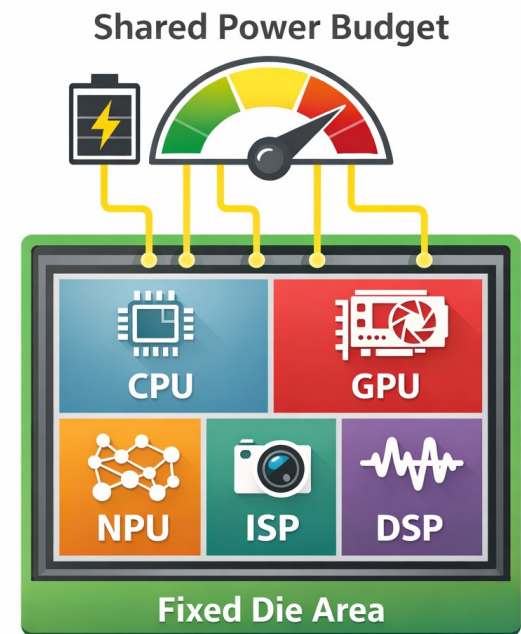
The Challenge: Big Models, Small Devices

- Devices have **strict size/weight** constraints
- There isn't much room or power budget, so the battery, board area, and cooling options are all limited.
- **Result:** compute, memory, energy, and thermals cannot scale like servers



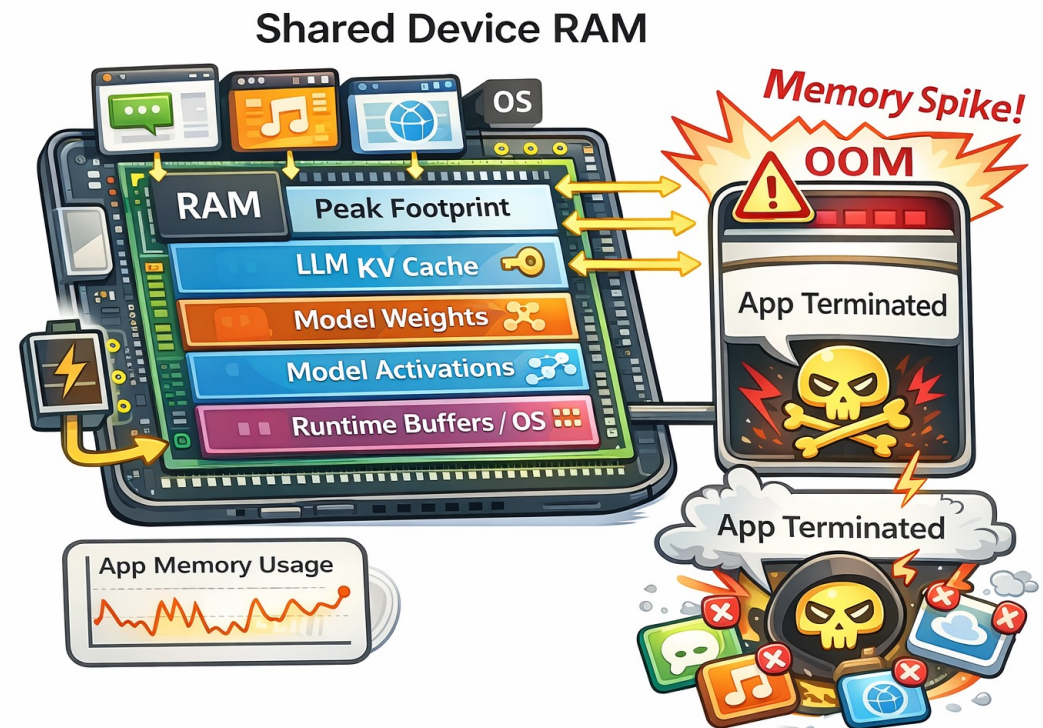
Limited Compute: Efficiency Over Peak Performance

- SoCs have fixed die area and a strict power budget shared by various processors
 - SoC (System on Chip): Multiple components integrated into a single chip
 - CPU— Central Processing Unit
 - GPU — Graphics Processing Unit
 - NPU — Neural Processing Unit
 - ISP — Image Signal Processor
 - DSP — Digital Signal Processor
- More compute increases area and power, raising heat and throttling risk
- **"Dark Silicon"**: not all transistors can run at once within thermal limits
- Practical takeaway: peak Tera Operations Per Second (TOPS) is often misleading; sustained throughput and TOPS/W matter



Limited Memory: Capacity and Allocation

- Device RAM is limited and shared with the OS and other apps
 - apps face hard memory caps.
- Peak footprint includes model weights + model activations + runtime/OS buffers
 - + LLMs add KV cache that grows with context
- If memory spikes past limits, the OOM killer can terminate the app instantly



Limited Energy: Strict Power Envelopes

- **Battery Limits for All-Day Use:** Unlike cloud servers, on-device AI must operate within a strictly limited battery capacity
 - The primary goal is to minimize Energy per Inference
- **System-Wide Power Contention:** AI cannot monopolize the power budget. It must share limited energy with essential components like the display, network radios, sensors, and OS tasks
- **The High Cost of Data Movement:** Fetching data from memory (DRAM) consumes significantly more energy than actual computation (ALU)
 - Goal: Maximize data locality and minimize memory access to save battery.



Limited Thermals: Sustained Performance

- **Strict Thermal Constraints:** Passive cooling leaves minimal thermal headroom
 - Device surface temperatures must be capped ($\sim 45^{\circ}\text{C}$) to prevent discomfort **and low-temperature burns**
- **Thermal Throttling:** Reaching temperature limits forces the system to reduce CPU/GPU/NPU clock speeds
 - This causes sudden latency spikes and jitter during sustained workloads
- **Impact on On-Device UX:** A smooth user experience relies on stable tail latency, rather than just high average processing speed



Some Important Metrics

- **Latency (ms)**
 - End-to-end time to process a single inference request
 - Critical for interactive applications
 - Touch-response latency above ~50 ms feels laggy to users
- **Throughput (inferences per second)**
 - Number of inference requests processed per second
 - For video applications, often expressed as FPS
- **Tail Latency (99th percentile latency)**
 - Measures worst-case latency rather than average
 - Occasional spikes degrade user experience
- **Peak Memory**
 - Peak memory = maximum RAM used at any single moment
 - Includes
 - Model weights
 - Intermediate activations
 - Temporary buffers
 - Runtime and OS overhead
 - The highest spike determines whether the app survives

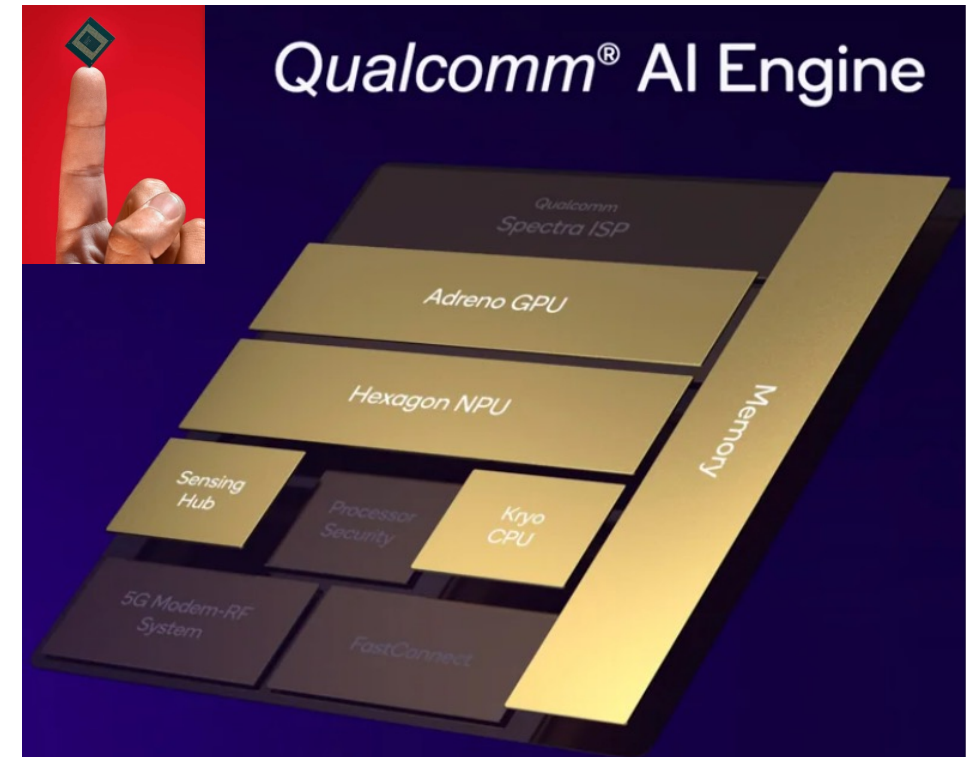
On-Device AI Hardware

Why Hardware Matters in On-Device AI

- On-device AI operates under strict physical constraints
 - Battery-powered devices
 - Strict thermal limits (~45°C surface temperature)
 - Shared system power budget (display, modem, sensors, radio)
 - Limited memory bandwidth
 - Real-time interaction requirements
- Cloud AI scales with more GPUs
- On-device AI must optimize within fixed hardware
- **Key message:** On-device AI is a constrained systems problem, not only a modeling problem

What Is Inside a Smartphone SoC

- Modern smartphones integrate multiple components in one SoC (System-on-Chip)
 - **CPU** for general-purpose control
 - **GPU** for parallel compute and graphics
 - **NPU** or AI Engine for neural acceleration
 - ISP for camera processing
 - DSP for low-power signal processing
 - On-chip SRAM
 - External DRAM
- All components share
 - Power budget
 - Thermal envelope
 - Memory hierarchy
- AI performance depends heavily on how these components interact



Qualcomm Snapdragon 8 Gen 3

CPU, GPU, NPU: Why All Three

Processor	Design Focus	Strength	Limitation	Role in On-Device AI
CPU	General-purpose control	• Flexible execution • Handles branching and irregular ops	Low energy efficiency for dense tensor math	• Orchestration • Pre/post-processing
GPU	Parallel throughput	• Massive parallel FP16 compute • High raw throughput	• Higher power consumption • Shared with graphics	• Medium-size models • Fallback execution
NPU	Energy-efficient tensor ops	• High TOPS per watt • Optimized for INT8 / INT4	Limited operator flexibility	Primary inference engine

- **AI accelerator:** is a general term for hardware specialized for AI computation (NPU, TPU, and GPU if it's used for AI)
- **TPU:** is Google's specific implementation of an AI accelerator

What Makes NPUs Efficient

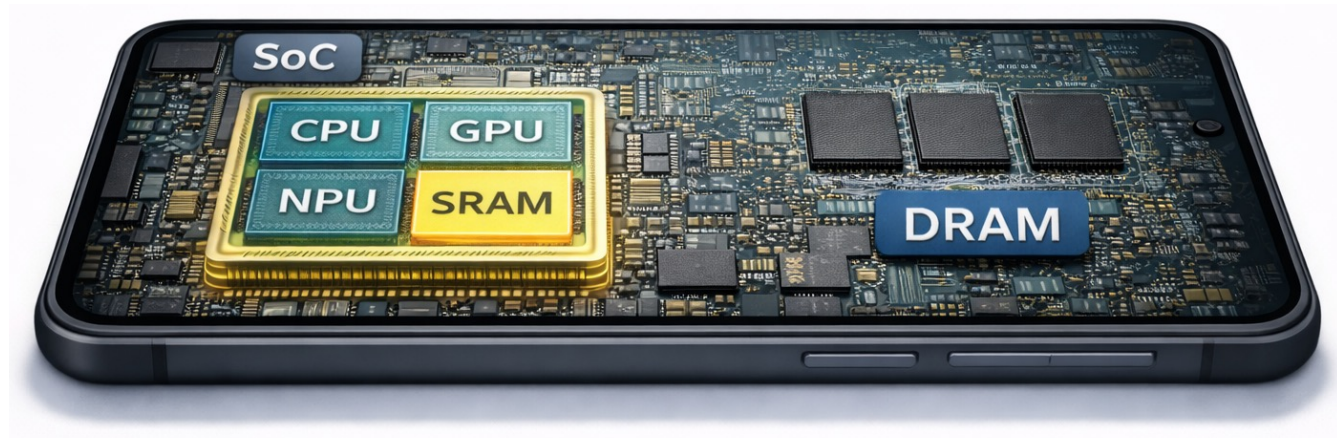
- NPUs are designed for **tensor**(multi-dimensional array)-centric computation
 - Massive parallel MAC (Multiply–Accumulate; $y=y+(a \times b)$) units
 - Structured dataflow
 - Operator specialization such as Conv and GEMM (General Matrix–Matrix Multiplication)
 - Reduced control overhead
- Common hardware terms
 - Systolic array (minimizing memory access bottlenecks)
 - TOPS (Tera Operations Per Second)
 - TOPS per watt
- Core idea
 - Not maximum speed, but maximum energy-efficient tensor execution

Qualcomm
Hexagon NPU



Memory Hierarchy: SRAM vs DRAM

- Memory is often the real bottleneck in on-device AI
- **On-chip SRAM**
 - Small capacity
 - Very fast
 - Energy-efficient
 - Used as working memory or scratchpad
- **External DRAM**
 - Large capacity
 - Slower access
 - High energy cost per access
 - Shared across system components
- Energy cost of DRAM access is far higher than a single MAC operation
- Implications
 - Keep working sets inside SRAM
 - Use tiling to fit tensors into on-chip memory
 - Reduce unnecessary DRAM round-trips
- Model structure directly affects memory behavior



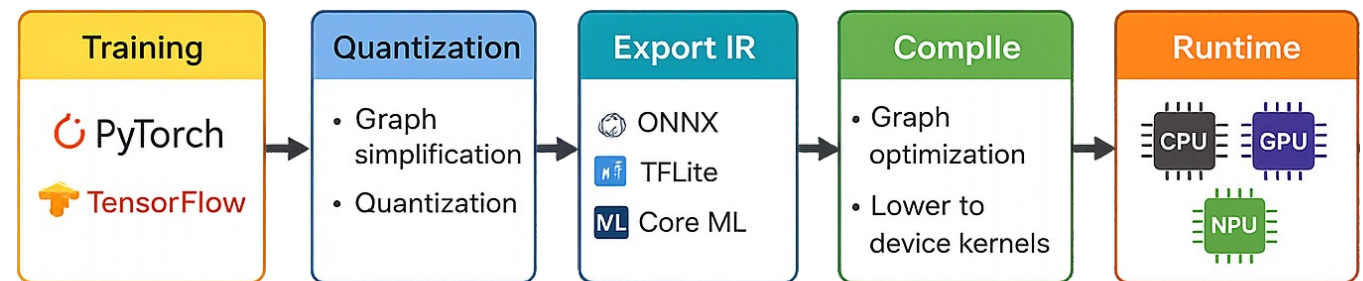
On-Device AI as a System Pipeline

- Real applications execute as pipelines
 - Camera → ISP → NPU → CPU → Display
 - Microphone → DSP → NPU → CPU
- System-level concepts
 - Unified memory
 - Buffer reuse
 - Zero-copy execution
 - System-level cache
- On-device AI is **coordinated execution across multiple engines**; not a single model running on a single accelerator

On-Device AI Software Ecosystem

Big Picture: From Training to On-Device Execution

- **Typical end-to-end pipeline:**
 1. Train model in PyTorch or TensorFlow
 2. Prepare for deployment
 - graph simplification
 - quantization
 3. Export to an interchange format
 - ONNX
 - TFLite
 - Core ML format
 4. Compile for the target device
 - graph optimization
 - lowering to device kernels
 5. Run on device
 - runtime loads and executes
 - OS delegates to CPU, GPU, NPU
 - vendor drivers launch hardware

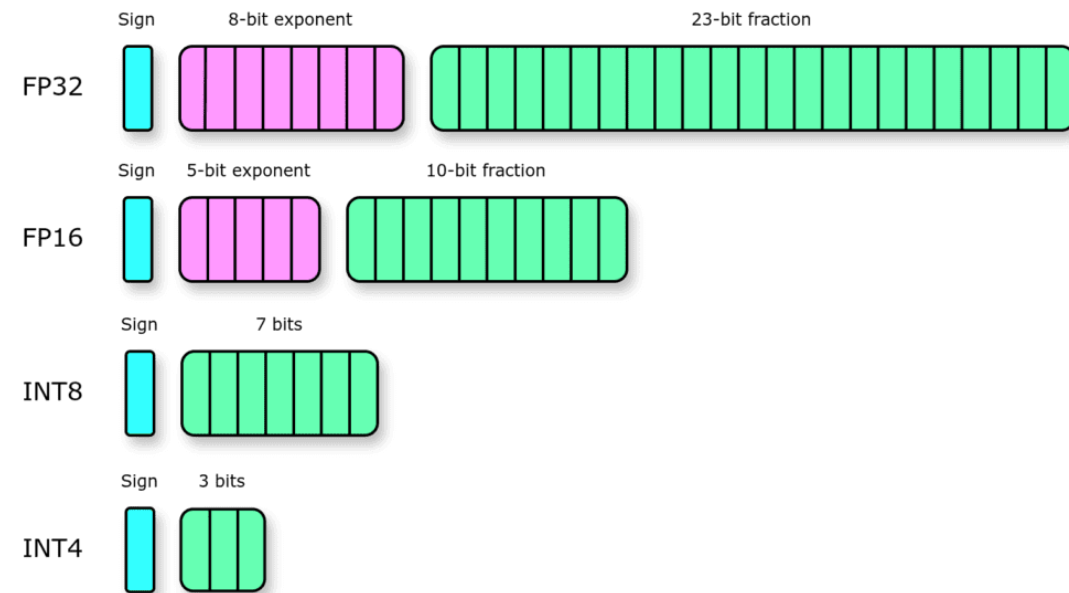


Why This Pipeline Exists

- The compatibility headache
- Raw training checkpoints/weights cannot run directly on phone NPUs
- Differences across devices
 - OS APIs and runtimes
 - SoC vendors and driver stacks
 - supported operators and data types
- Training is framework-centric, but deployment is hardware-centric
- Training optimizes accuracy, while deployment optimizes feasibility and efficiency

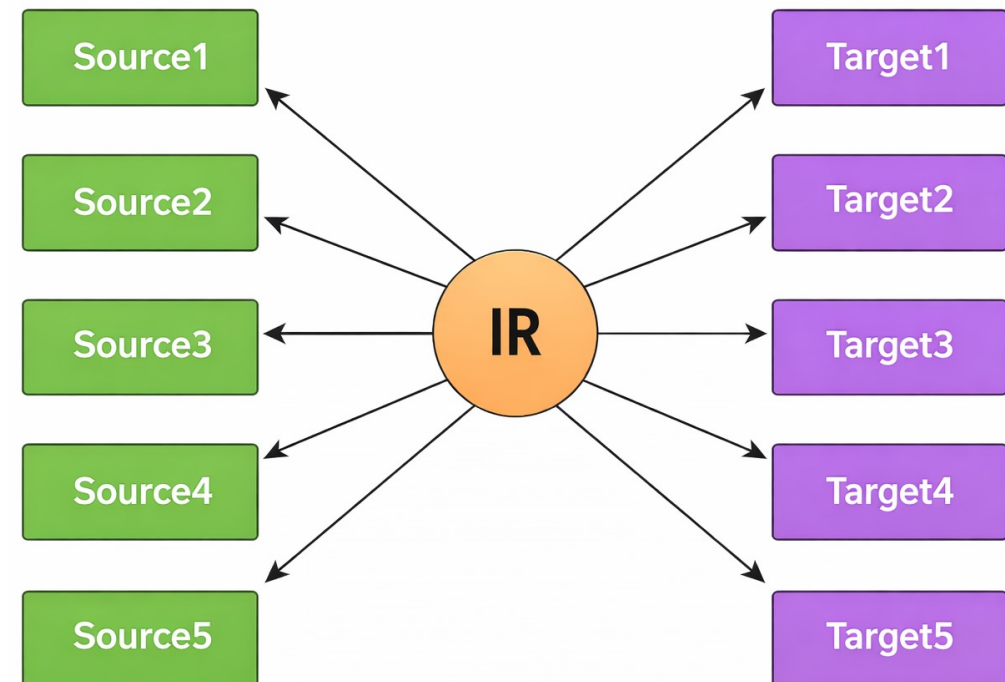
Deployment Preparation: Quantization and Graph Constraints

- Integer-first reality
 - Many NPUs are fastest on INT8
 - To stay on the NPU fast path, the model must look like an INT8-friendly graph
 - FP16 may work but often less efficient
 - FP32 often inefficient or unsupported
- Graph preparation
 - Fold or fuse common patterns to match accelerator kernels
 - Conv → BN → ReLU (Conv-BN fusion)
 - Remove training-only ops that cannot be deployed
 - dropout
 - Rewrite to a supported operator set to avoid CPU and GPU fallback
- Quantization
 - Converts model weights from higher precision to lower precision
 - e.g., FP16 → INT8
 - Often quantizes activations as well to keep execution in integer
 - Reduces model size and matches accelerator-supported data types
- Goal
 - smaller model and memory footprint
 - lower bandwidth and energy
 - more operators run on the NPU



Export: IR as the Bridge from Training to Deployment

- **Intermediate Representation (IR)** is the portable model representation
 - serialized computation graph
 - operators and tensor shapes
 - data types and weights
- Examples
 - ONNX
 - TFLite FlatBuffer
 - Core ML format
- Why it matters
 - decouple training frameworks from deployment toolchains
 - enable compilation without Python runtime
 - avoid bundling large ML frameworks on devices



Compilation: Turning IR Into Efficient On-Device Code

- The model becomes a compiled program
- Compiler passes
 - operator fusion
 - constant folding
 - dead code elimination
 - layout transformation
 - memory planning and buffer reuse
- Outcome
 - fewer intermediate tensors written to DRAM
 - better SRAM reuse
 - fewer kernel launches
 - faster and more energy efficient execution

Runtime and Delegation: CPU-GPU-NPU Cooperation

- **Not everything runs on the NPU**
 - NPUs are optimized for some operations (CNN, GEMM, etc.)
- Runtime responsibilities
 - load compiled model
 - allocate buffers and manage execution
 - provide predictable performance
- Delegation
 - Android NNAPI
 - iOS Core ML
- Placement depends on
 - operator support
 - available accelerators
 - system state such as thermal and power
- Graph may be partitioned across CPU, GPU, NPU
- Data movement between engines has cost

Wrap-Up: What Software Enables

- On-device AI software stack enables
 - portability across devices
 - hardware abstraction
 - integer-efficient execution via quantization
 - compiler optimizations for memory and latency
 - heterogeneous execution across CPU, GPU, NPU

Software Is Our Leverage!

- Hardware defines budget, while **software decides how wisely we spend it**
- As CSE students, we design
 - Efficient AI algorithms
 - Model architectures
 - Compression and quantization strategies
 - Memory layouts
 - Execution scheduling
 - Runtime systems and compilers
- Hardware awareness enables
 - More deployable research
 - More realistic system design
 - Stronger impact in on-device AI
- Takeaway
 - On-device AI = Model + Software + Hardware co-design
 - Our advantage is intelligent software design guided by hardware understanding



Be Ready for the Upcoming Lectures

- Starting from the next lecture, I will assume that you are familiar with the following concepts:
 - **Basics of Deep Learning**
 - training, inference, loss, datasets, etc.
 - **How Transformers work**
 - **How LLMs work**
- If these concepts are unfamiliar to you, please review them in advance through online resources (e.g., Coursera deep learning courses, YouTube lectures, etc.)